CS405 Project 1 Report

Elif Sude Yanar, 32431
Painting: Şeker Ahmed Paşa, Kavunlu Natürmort

1. Painting

I chose Kavunlu Natürmort because the melon has a clear and smooth outline, so it is good for tracing a curve. Since it is a still life on a table, I also put a table with some objects in front of the painting and a smallhanging mobile above it. The painting is a textured quad on the wall behind them.

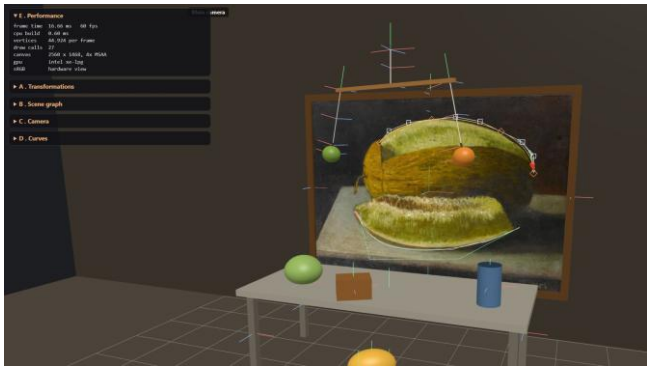


Figure 1: the scene from the main camera



Figure 2: Part B, node axes on, tree and world(fruitL) in the panel

2. Part A - Transformations

The matrix functions are in src/math/mat4.js. Matrices are column major so they can be written to the uniform buffer directly. For the inverse of a rigid transform I used $R^{-1} = R^T$, so the inverse of [R t] is $[R^T \; -R^T t]$.

The orange melon is controlled from the GUI. With T = T(0.45, 1, -0.42), R = R_y(0.6) and S = S(0.3) the two orders give:

```
T*R*S                       S*R*T
 0.25   0.00   0.17   0.45   0.25   0.00   0.17   0.04
 0.00   0.30   0.00   1.00   0.00   0.30   0.00   0.30
-0.17   0.00   0.25  -0.42  -0.17   0.00   0.25  -0.18
 0.00   0.00   0.00   1.00   0.00   0.00   0.00   1.00
```

The rotation and scale part is the same because the scale is uniform, only the last column changes. In T*R*S the object is scaled and rotated around its own origin first and then moved, so it ends up at the translation I gave. In S*R*T the translation happens first, then it gets rotated around the world origin and scaled by 0.3, so the translation becomes S*R*t = (0.04, 0.30, -0.18) and the melon falls under the table. I use T*R*S in the rest of the project because then translation only decides where the object is.

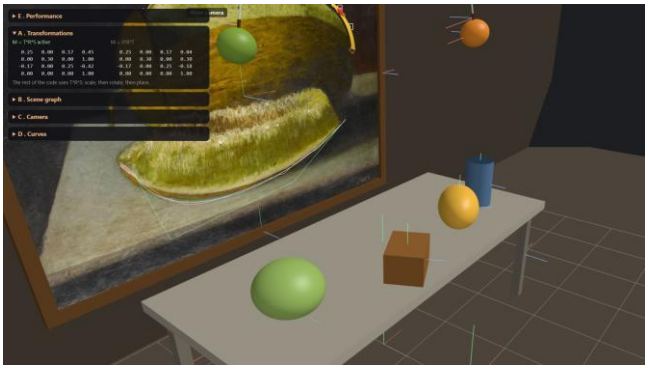


Figure 3: T*R*S

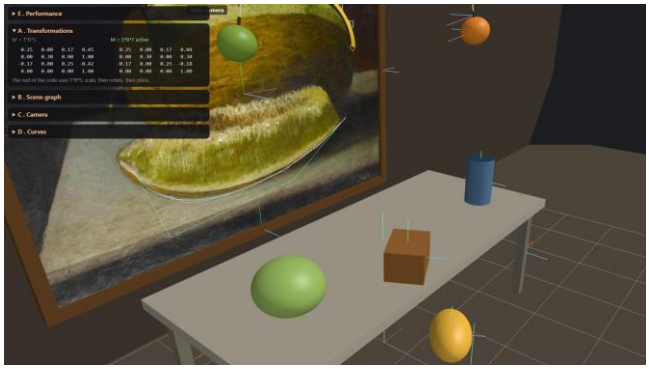


Figure 4: S*R*T, melon is on the floor

3. Part B - Scene graph

Each node has a local matrix, a parent and children. The world matrix is computed recursively as world = parentWorld * local. The mobile is the animated part:

```
mobile        T(0, 2.95, -0.75)
  crossbar    R_y(spin)                      spin = 0.6t
    armL      T(-0.6, -0.3, 0) * R_z(-swing)  swing = 0.35 sin(1.7t)
      fruitL  T(0, -0.62, 0)
        stemL
    armR      T(0.6, -0.3, 0) * R_z(swing)
      fruitR
        stemR
```

There are also mesh nodes like the strings, the bar and the fruit shapes. The deepest level is 5. Figure 2 shows the local axes of every node.

Checking fruitL by hand at t = 1.3: spin = 0.78 so cos = 0.711 and sin = 0.703. swing = 0.35 sin(2.21) = 0.281 so cos = 0.961 and sin = 0.277. The first column of $R_y(0.78)R_z(-0.281)$ is $(\cos_y\cos_z, -\sin_z, -\sin_y\cos_z)$ = (0.683, -0.277, -0.676). For the position, $R_z(-0.281)$ turns (0, -0.62, 0) into (-0.172, -0.596, 0). Adding the arm offset gives (-0.772, -0.896, 0), R_y turns this into (-0.549, -0.896, 0.543) and adding the mobile position gives (-0.549, 2.054, -0.207). The program prints the same:

```
world(fruitL)
 0.683  0.197  0.703 -0.549
-0.277  0.961  0.000  2.054
-0.676 -0.195  0.711 -0.207
 0.000  0.000  0.000  1.000
```

4. Part C - Camera and projection

View matrix

The camera looks along -z. From the eye e, target c and up:

w = normalize(e - c), u = normalize(up × w), v = w × u

The camera's world matrix is [u v w e]. It is a rigid matrix so its inverse is easy, and that is the view matrix. The rows are u, v, w and the last column is (-u·e, -v·e, -w·e). This moves the eye to the origin and the target onto the -z axis.

If up is parallel to w, up × w is zero and cannot be normalized. In that case I use another axis, (0, 0, -1) or (0, 1, 0), instead of up. View 4 puts the camera right above the table to test this, and the matrix stays valid.

Projection

WebGPU wants depth in [0, 1]. With $w_{clip} = -z$ and $z_{clip} = Az + B$, I need z = -n to give 0 and z = -f to give 1 after dividing by w:

-An + B = 0 and -Af + B = f, so A = f / (n - f) and B = nf / (n - f).

x and y are multiplied by cot(fov/2)/aspect and cot(fov/2). The orthographic matrix uses $z_{clip} = z/(n - f) + n/(n - f)$ with no divide. Its height is set from the distance to the target so switching does not change the size too much.

Near and far

I used near = 0.1 and far = 8. The whole room fits in 8 units from the default camera. After the divide the depth is $d(z) = f(z + n) / ((f - n)z)$, which is not linear: $d(-2n)$ is about 0.5, so half of the depth values are used between n and 2n. With far/near = 80 there is enough precision for the table and the painting. If near was 0.001, half of the depth range would be used in the first 2 mm in front of the camera, so it is better to keep near as large as possible.

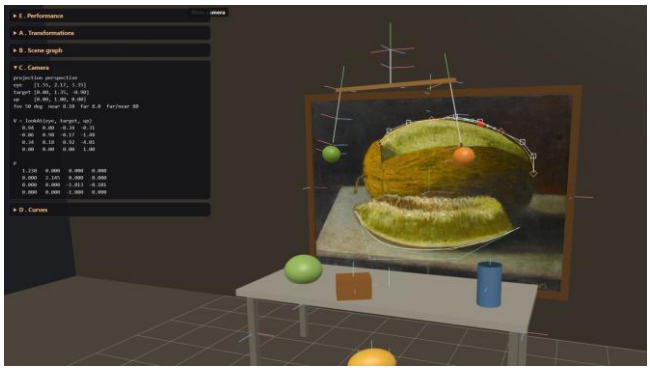


Figure 5: perspective

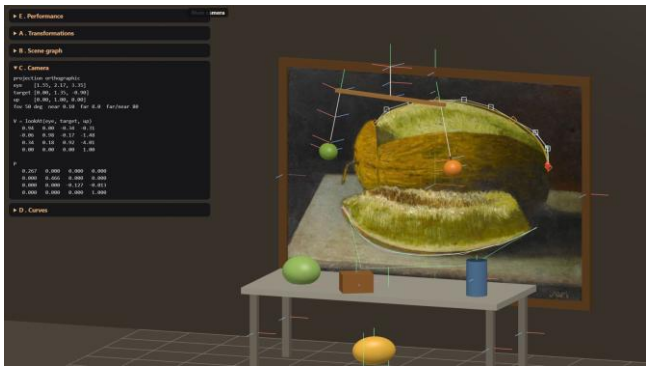


Figure 6: orthographic

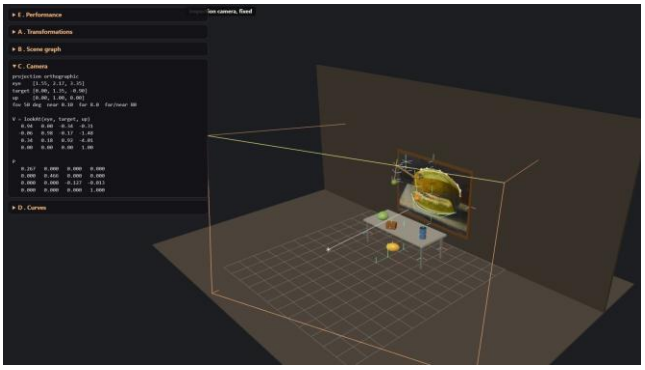


Figure 7: frustum of the main camera seen from the inspection camera

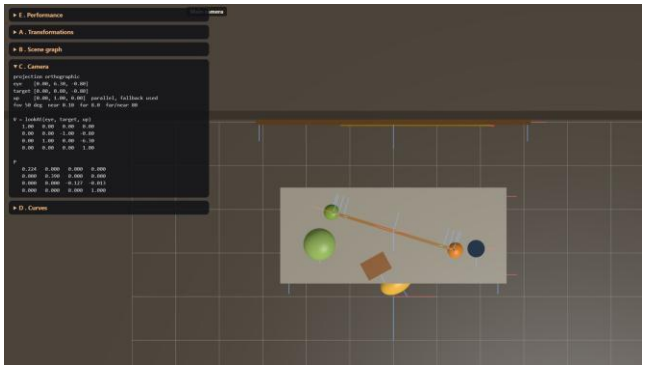


Figure 8: view 4, up parallel to the view direction

The frustum is drawn by taking the 8 corners of the NDC cube and multiplying them with inverse(P*V). This works for both projections.

5. Part D - Curves

Cubic Bezier: $B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$

The four basis functions are always positive on [0, 1] and they add up to $((1-t) + t)^3 = 1$.

Start and end points: At t = 0 every term except the first one has a t in it, so $B(0) = P_0$. At t = 1 every term except the last one has a (1-t) in it, so $B(1) = P_3$.

C1 at a joint: The derivative is $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$. If segment A ends at knot K and segment B starts at K, then for C1 we need $3(K - A_2) = 3(B_1 - K)$, so $B_1 = 2K - A_2$. The handles have to be on opposite sides of the knot with the same length. When you drag a handle the program moves the other handle to keep this.

My curve follows the top of the melon with 3 segments. de Casteljau is drawn for the current t. For the second curve type I used Catmull-Rom, which goes through the 4 knots. The red arrow moves along the curve and turns with the tangent: its x axis is the tangent and its z axis is the painting normal.



Figure 9: traced curve with de Casteljau

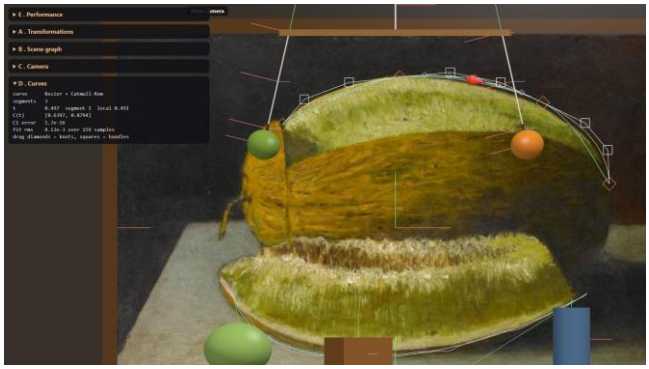


Figure 10: Bezier in yellow, Catmull-Rom in blue

6. Bonus

- Least squares fit: shift + drag draws a stroke. P_0 and P_3 are the first and last points, the parameters start with chord length and P_1, P_2 come from a 2x2 linear system. Then a few Newton steps fix the parameters and it is solved again. For the bottom of the melon slice with 159 points the RMS error was 0.0081.
- Surface of revolution: the traced curve is used as a profile and turned around the y axis. It is the green object on the table and it changes when the curve is edited.
- Quaternion camera: the saved views use slerp so the camera turns smoothly.
- Compute shader: the curve is evaluated on the GPU and written as a ribbon into a vertex buffer.



Figure 11: stroke and the fitted curve

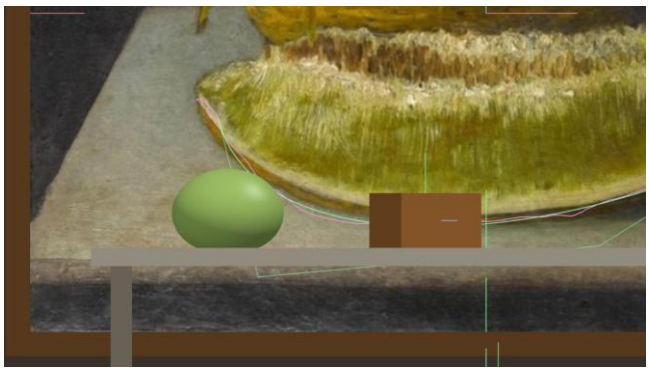


Figure 12: surface of revolution

7. Performance

Measured in Chrome 156 on my laptop, integrated Intel Graphics, Xe-LPG, driver 32.0.101.8332, canvas 2560 x 1468, 4x MSAA.

View	Vertices per frame	Draw calls	Frame time	CPU frame build
Split	88,510	55	16.66 ms, 60 fps	0.60 ms
Main only	44,238	27	17.55 ms, 60 fps	0.52 ms
Inspection only	44,272	28	16.65 ms, 60 fps	0.50 ms

The frame time is one 60 Hz refresh because requestAnimationFrame waits for vsync, so the GPU is not the limit and the CPU build time is the more useful number. Split view draws everything once per camera, so it doubles the counts. The inspection view has one extra draw call for the frustum overlay.

8. Problems

- The first curve points did not follow anything in the painting, they crossed through the melon. This was found by drawing the curve on top of the image, and the knots were moved to the top edge of the melon.
- The least squares test failed at first because 4 Newton iterations were not enough, the error was 0.0067. It was converging, just slowly, so it was changed to 20 iterations.
- Colors looked wrong if lighting is done in sRGB, so the canvas uses an sRGB view and lighting is done in linear space.
- On my laptop only the GUI was visible, but t kept changing, so the app was running. The CSS gave the error box display: grid, which overrides hidden, so an empty box covered the canvas. #error[hidden] { display: none; } fixed it.

Appendix A - Curve JSON


```
{
  "painting": {
    "artist": "Seker Ahmed Pasa",
    "title": "Kavunlu Naturmort"
  },
  "coordinates": "normalised painting coordinates, u to the
right and v up, both in [0, 1]",
  "type": "piecewise cubic Bezier",
  "segments": 3,
  "continuity": "C1",
  "c1Error": 1.67e-16,
  "controlPoints": [
    {
      "u": 0.297,
      "v": 0.7652,
      "role": "knot"
    },
    {
      "u": 0.3364,
      "v": 0.8304,
      "role": "handle"
    },
    {
      "u": 0.4167,
      "v": 0.8739,
      "role": "handle"
    },
    {
      "u": 0.5076,
      "v": 0.887,
      "role": "knot"
    },
    {
      "u": 0.5985,
      "v": 0.9001,
      "role": "handle"
    },
    {
      "u": 0.6894,
      "v": 0.8739,
      "role": "handle"
    },
    {
      "u": 0.7652,
      "v": 0.8217,
      "role": "knot"
    },
    {
      "u": 0.841,
      "v": 0.7695,
      "role": "handle"
    },
    {
      "u": 0.8818,
      "v": 0.7,
      "role": "handle"
    },
    {
      "u": 0.8848,
      "v": 0.613,
      "role": "knot"
    }
  ],
  "catmullRomKnots": [
    {
      "u": 0.297,
      "v": 0.7652
    },
    {
      "u": 0.5076,
      "v": 0.887
    },
    {
      "u": 0.7652,
      "v": 0.8217
    },
    {
      "u": 0.8848,
      "v": 0.613
    }
  ],
  "leastSquaresFit": {
    "control": [
      {
        "u": 0.8451,
        "v": 0.3315
      },
      {
        "u": 0.624,
        "v": 0.1347
      },
      {
        "u": 0.2501,
        "v": 0.0862
      },
      {
        "u": 0.2065,
        "v": 0.3339
      }
    ],
    "rms": 0.008126
  }
}
```


The full file is curve.json in the repository.

Appendix B - AI use

During the development of this project, Claude was a thought partner. Suggestions were taken regarding general code structure and formatting, and Claude was consulted on how to structure and execute the unit tests. The report itself was written independently, but Claude assisted in gathering necessary materials, organizing data, and preparing visual assets and obtaining screenshots (I had the screenshots but Claude guided me) used throughout the document.